

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 13-Sorting



Arsyaghali Nafiras

109082500068

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Algoritma Selection Sort

Mencari nilai ekstrem (terkecil atau terbesar) di dalam sekumpulan data yang tersisa, kemudian menukarnya ke posisi yang seharusnya.

2. Algoritma Insertion Sort

Menyisipkan suatu nilai secara langsung pada posisi yang tepat di dalam bagian data yang sudah terurut.

B. Guided

1. Guided-1

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {

    /* I.S. terdefinisi array T yang berisi n bilangan bulat

       F.S. array T terurut secara mengecil atau descending dengan INSERTION
       SORT */

    var temp, i, j int

    i = 1

    for i <= n-1 {

        j = i
```

```
temp = T[j]

for j > 0 && temp > T[j-1] {

    T[j] = T[j-1]

    j = j - 1

}

T[j] = temp

i = i + 1

}

}
```

```
func main() {

    var data arrInt

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        fmt.Scan(&data[i])

    }

    insertionSort(&data, n)
```

```

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

5
1 9 4 6 2
9 6 4 2 1

```

2. Guided-2

```

package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
    }
}

```

```
temp = T[j]

for j > 0 && temp < T[j-1] {

    T[j] = T[j-1]

    j = j - 1

}

T[j] = temp

i = i + 1

}

}
```

```
func main() {

    var data arrString

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        fmt.Scan(&data[i])

    }

    insertionSortString(&data, n)
```

```

    for i := 0; i < n; i++ {

        fmt.Print(data[i], " ")

    }

}

```

Output :

```

5
1 5 2 9 7
1 2 5 7 9

```

3. Guided-3

```

package main
import "fmt"

func selectionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

```

fmt.Println("\n=== SETELAH DISORITNG ===")
selecetionSortArray(&arrAngka)
for i := 0; i < len(arrAngka); i++ {
    fmt.Print(arrAngka[i], " - ")
}
}

```

Output:

```

Masukkan data angka ke-0 : 5
Masukkan data angka ke-1 : 8
Masukkan data angka ke-2 : 3
Masukkan data angka ke-3 : 6
Masukkan data angka ke-4 : 9

=== SEBELUM DISORTING ===
5 - 8 - 3 - 6 - 9 -
=== SETELAH DISORITNG ===
3 - 5 - 6 - 8 - 9 -

```

4. Guided-4

```

package main
import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK      float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa
}

```

```

for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
    fmt.Print("Nama : ")
    fmt.Scan(&structMHS[i].nama)
    fmt.Print("NIM : ")
    fmt.Scan(&structMHS[i].nim)
    fmt.Print("IPK : ")
    fmt.Scan(&structMHS[i].IPK)
}
fmt.Println()

fmt.Println(" === SEBELUM SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()

SelectionSortArray(&structMHS)
fmt.Println(" === SETELAH SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()

```

Output:

--- Masukkan Data Mahasiswa Ke-0 ---

Nama : lala

NIM : 152625

IPK : 3.1

--- Masukkan Data Mahasiswa Ke-1 ---

Nama : takumi

NIM : 785452

IPK : 3.2

--- Masukkan Data Mahasiswa Ke-2 ---

Nama : buahl

NIM : 95625

IPK : 2.0

--- Masukkan Data Mahasiswa Ke-3 ---

Nama : madara

NIM : 2.3

IPK : 2.0

--- Masukkan Data Mahasiswa Ke-4 ---

Nama : kohaku

NIM : 845454

IPK : 3.0

=== SEBELUM SORITNG ===

--- Data Mahasiswa Ke-0 ---

Nama : lala

NIM : 152625

IPK : 3.10

--- Data Mahasiswa Ke-1 ---

Nama : takumi

NIM : 785452

IPK : 3.20

--- Data Mahasiswa Ke-2 ---

Nama : buahl

NIM : 95625

IPK : 2.00

--- Data Mahasiswa Ke-3 ---

Nama : madara

NIM : 2.3

IPK : 2.00

--- Data Mahasiswa Ke-4 ---

Nama : kohaku

NIM : 845454

IPK : 3.00

=====

=== SETELAH SORITNG ===

--- Data Mahasiswa Ke-0 ---

Nama : buahl

NIM : 95625

IPK : 2.00

--- Data Mahasiswa Ke-1 ---

Nama : madara

NIM : 2.3

IPK : 2.00

--- Data Mahasiswa Ke-2 ---

Nama : kohaku

NIM : 845454

IPK : 3.00

--- Data Mahasiswa Ke-3 ---

Nama : lala

NIM : 152625

IPK : 3.10

--- Data Mahasiswa Ke-4 ---

Nama : takumi

NIM : 785452

IPK : 3.20

=====

C. Unguided

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masingmasing daerah.

```
package main

import "fmt"

func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        houses := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&houses[j])
        }
    }
}
```

```

        selectionSort(houses)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(houses[j])
        }
        fmt.Println()
    }
}

```

Output :

```

3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program mencari tahu berapa banyak daerah yang harus diproses lalu ulangi per Daerah, yang mana untuk setiap daerah program akan melakukan tiga hal yaitu membaca jumlah rumah dan menyimpan semua nomor rumahnya ke dalam daftar, program mencari nomor rumah terkecil dan menaruhnya di depan lalu mengulangi proses tersebut untuk angka sisa lainnya hingga urut membesar, dan yang terakhir menampilkan nomor rumah yang sudah rapi ke layar, lalu pindah baris untuk daerah berikutnya.

- 2) Belakang diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format Masukan masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

```

package main

import (
    "fmt"
)

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomorRumah int
            fmt.Scan(&nomorRumah)

            if nomorRumah%2 != 0 {
                ganjil = append(ganjil, nomorRumah)
            } else {
                genap = append(genap, nomorRumah)
            }
        }

        for x := 0; x < len(ganjil); x++ {
            for y := x + 1; y < len(ganjil); y++ {
                if ganjil[x] > ganjil[y] {
                    ganjil[x], ganjil[y] = ganjil[y],
ganjil[x] // Tukar posisi
                }
            }
        }

        for x := 0; x < len(genap); x++ {
            for y := x + 1; y < len(genap); y++ {
                if genap[x] < genap[y] {
                    genap[x], genap[y] = genap[y], genap[x]
                }
            }
        }

        hasil := append(ganjil, genap...)

        for k, val := range hasil {
            if k > 0 {
                fmt.Print(" ")
            }
        }
    }
}

```

```

        fmt.Print(val)
    }
    fmt.Println()
}
}

```

Output :

```

3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program membaca berapa banyak baris daerah yang akan diproses lalu di setiap baris, program membaca angka-angka rumah lalu memisahkannya ke dalam dua wadah yaitu wadah ganjil dan wadah genap. Yg mana jika Wadah ganjil diurutkan dari kecil ke besar dan Wadah genap diurutkan dari besar ke kecil. Lalu ganjil dan genap digabungkan, lalu hasilnya langsung dicetak.

- 3) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

```

package main

import "fmt"

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1
    }
}

```

```

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j = j - 1
        }
        arr[j+1] = key
    }
}

func main() {
    var arr []int
    var input int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        arr = append(arr, input)
    }

    if len(arr) == 0 {
        return
    }
    insertionSort(arr)

    for i, val := range arr {
        fmt.Print(val)
        if i < len(arr)-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    if len(arr) < 2 {
        fmt.Println("Data berjarak 0")
        return
    }

    selisihAwal := arr[1] - arr[0]
    isTetap := true

    for i := 1; i < len(arr)-1; i++ {
        selisihSekarang := arr[i+1] - arr[i]
        if selisihSekarang != selisihAwal {
            isTetap = false

```

```

        break
    }
}

if isTetap {
    fmt.Printf("Data berjarak %d\n", selisihAwal)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output:

```

31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\ALPRO-Praktikum_semester-2\
8_Arsyaghali-Nafiras\Week 12 - mo
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan:

Program membaca angka satu per satu. Angka ≥ 0 disimpan, sedangkan angka < 0 menghentikan proses input lalu Angka yang tersimpan diurutkan dari terkecil ke terbesar menggunakan metode Insertion Sort dan program akan memberikan dua output yaitu mencetak angka-angka yang sudah rapi berurutan dalam baris pertama dan mengecek selisih antar angka. Jika selisihnya selalu sama maka akan memberikan output "Data berjarak x" dan Jika ada selisih yang berbeda maka outputnya "Data berjarak tidak tetap".

- 4) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan.

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

```

package main

import "fmt"

const nMax = 7919

type Buku struct {
    id      string
    judul   string
    penulis string
    penerbit string
    eksemplar int
    tahun   int
    rating  int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n > 0 {
        maxRating := pustaka[0].rating
        idxMax := 0
        for i := 1; i < n; i++ {
            if pustaka[i].rating > maxRating {
                maxRating = pustaka[i].rating
                idxMax = i
            }
        }
        fmt.Printf("%s %s %s %d\n", pustaka[idxMax].judul,
pustaka[idxMax].penulis, pustaka[idxMax].penerbit, pustaka[idxMax].tahun)
    }
}

```



```

}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i - 1

        for j >= 0 && pustaka[j].rating < temp.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    batas := 5
    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri := 0
    kanan := n - 1
    ketemu := false
    tengah := 0

    for kiri <= kanan && !ketemu {
        tengah = (kiri + kanan) / 2

        if pustaka[tengah].rating == r {
            ketemu = true
        } else if pustaka[tengah].rating < r {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if ketemu {
        fmt.Printf("%s %s %s %d %d %d\n",

```

```

        pustaka[tengah].judul,
        pustaka[tengah].penulis,
        pustaka[tengah].penerbit,
        pustaka[tengah].tahun,
        pustaka[tengah].eksemplar,
        pustaka[tengah].rating,
    )
} else {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
}
}

func main() {
    var Pustaka DaftarBuku
    var nPustaka int
    var rCari int

    DaftarkanBuku(&Pustaka, &nPustaka)
    CetakTerfavorit(Pustaka, nPustaka)
    UrutBuku(&Pustaka, nPustaka)
    Cetak5Terbaru(Pustaka, nPustaka)
    fmt.Scan(&rCari)
    CariBuku(Pustaka, nPustaka, rCari)
}

```

Output:

```

4
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 10 2005 5
B02 Bumi_Manusia Pramoedya_Ananta_Toer Lentera_Dipantara 5 1980 4
B03 Negeri_5_Menara A.Fuadi Gramedia 12 2009 3
B04 Ronggeng_Dukuh_Paruk Ahmad_Tohari Gramedia 8 1982 5
Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 2005
Laskar_Pelangi
Ronggeng_Dukuh_Paruk
Bumi_Manusia
Negeri_5_Menara
4
Bumi_Manusia Pramoedya_Ananta_Toer Lentera_Dipantara 1980 5 4

```

Penjelasan:

Membaca jumlah buku (N) beserta detail informasinya dan menyimpannya ke dalam array lalu mencetak 1 buku dengan rating tertinggi, kemudian mengurutkan seluruh data buku secara menurun (descending) menggunakan Insertion Sort dan mengoutput 5 besar judul buku dengan rating tertinggi, lalu mencari buku dengan rating tertentu menggunakan Binary Search.

D. Kesimpulan

I. Konsep Dasar Pencarian (Searching)

- Pencarian data dilakukan secara spesifik untuk menemukan elemen tertentu (seperti nama mahasiswa, NIM, atau alamat), berbeda dengan pencarian nilai ekstrem (minimum/maksimum) yang nilainya selalu pasti ditemukan.
- Pada kasus pencarian spesifik ini, terdapat dua kemungkinan hasil akhir: data yang dicari **ditemukan** atau **tidak ditemukan**.
- Algoritma pencarian dapat dimodifikasi untuk mengembalikan status keberadaan data (tipe *boolean*) maupun lokasi/indeks dari data tersebut di dalam array. Jika tidak ditemukan, biasanya fungsi akan mengembalikan nilai -1.

II. Algoritma Pencarian yang Dipelajari

- Sequential Search (Pencarian Sekuensial)
- Binary Search (Pencarian Biner)
- Pencarian pada Array of Struct